

23 分布式锁

- [修订记录](#)
- [1 配置 Redis 或 ZooKeeper](#)
- [2 注入分布式锁模板](#)
- [3 选择分布式锁实现方式](#)

修订记录

版本	日期	内容	作者
1.0	2020-8-17	初稿	胡冰

分布式锁用于在集群环境下确保一个方法在同一时刻只能被一个节点执行。只有获取到锁的节点才能执行方法，其他节点只能等待锁释放后才能继续尝试获取锁，执行指定的方法。

目前系统框架中的分布式锁提供了 ZooKeeper（基于 CuratorFramework 框架）和 Redis （基于 Redission 框架）两种实现方式，因此要使用分布式锁，系统中必须有 Redis 或者 Zookeeper 服

务器，二者只需要一个即可。

如果系统中同时有 Redis 和 Zookeeper，则默认使用 Redis，因为 Redis 比 ZooKeeper 更常用，且读写性能也要优于 ZooKeeper。也可以指定使用 ZooKeeper 作为实现分布式锁的 provider。

使用分布式锁的步骤如下。

1 配置 Redis 或 ZooKeeper

要使用分布式锁，必须先配置 Redis 或 ZooKeeper。

Redis:

```
# Redis
tiger:
  enable:
    redis: true

# Redis
spring:
  redis:
    cluster:
      nodes: '10.177.15.11:7000,10.177.15.12:7000,10.177.15.13:7000,10.177.15.14:7000,10.177.15.15:7000,10.177.15.16:7000'
```

ZooKeeper:

```
# Zookeeper
tiger:
  enable:
    zk: true

# ZooKeeper
tiger:
  zk:
    address: '10.213.120.11:2181,10.213.120.12:2181,10.213.120.13:2181'
```

详细配置请参考 Redis 和 ZooKeeper 开发指南。

2 注入分布式锁模板

系统提供分布式锁模板 **LockTemplate**，该模板提供如下接口：

```
/**
 *
 *
 */
public interface LockTemplate {

    /**
     *
     * <ol>
     *     <li></li>
     *     <li>callback</li>
     *     <li></li>
     * </ol>
     *
     * @param callback
     */
    void exec(LockCallback callback);
}
```

就一个 exec 方法，参数为一个回调接口 LockCallback，用于设置锁名称、超时时间和实际执行的操作：

```
/**
 *
 */
public interface LockCallback {

    /**
     *
     */
    String getLockName();

    /**
     * <=0
     * <p>10</p>
     */
    default int getTimeoutForAcquireLock() {
        return 10;
    }

    /**
     *
     */
    void action();
}
```

使用时，直接注入该模板即可：

```

@Service
public class UserServiceImpl implements UserService {

    /**
     * @Qualifier Redis
     */
    @Autowired
    private LockTemplate lockTemplate;

    public void sync() {
        //
        lockTemplate.exec(new LockCallback() {
            /**
             * ". "
             */
            @Override
            public String getLockName() {
                return "com.hngtrust.user.service.UserServiceImpl.sync";
            }

            /**
             *
             */
            public int getTimeoutForAcquireLock() {
                return 60;
            }

            /**
             *
             */
            @Override
            public void action() {
                //
            }
        });
    }
}

```

3 选择分布式锁实现方式

LockTemplate 有两种实现，分别是 Redis 和 ZooKeeper，适用于安装了 Redis 和 ZooKeeper 的场景。

1) 使用 Redis 实现

如果注入时不指定 @Qualifier，则默认使用 Redis 实现，因此：

```

@Autowired
private LockTemplate lockTemplate;

```

等价于：

```

@Autowired
@Qualifier("redisLockTemplate")
private LockTemplate lockTemplate;

```

2) 使用 ZooKeeper 实现

如果想使用 ZooKeeper 实现（比如系统中只有 ZooKeeper，没有 Redis），则在注入式指定 @Qualifier("zkLockTemplate")：

```

@Autowired
@Qualifier("zkLockTemplate")
private LockTemplate lockTemplate;

```